66.7
Pass

# Rakesh Narayan Tashildar

Other

*PDF generated at: 2 Mar 2026 18:55:46 UTC*

View this report on HackerRank

## Score

66.7% • 150.0 / 225

scored in eClerx - FO Java, Unix & Python Assessment in 83 min 52 sec on 2 Mar 2026 22:59:37 IST

## Candidate Information

| | |
|---|---|
| Email | tashildar@zohomail.in |
| Test | eClerx - FO Java, Unix & Python Assessment |
| Candidate Packet | View |
| Taken on | 2 Mar 2026 22:59:37 IST |
| Time taken | 83 min 52 sec/ 90 min |
| Work Experience | 3 years |
| Invited by | Priyanshi |
| Invited on | 2 Mar 2026 17:48:37 IST |

## Skill Distribution

| No. | Skill | Score |
|---|---|---|
| 1 | Bash<br>Intermediate | 0% |

| | | |
|---|---|---|
| 2 | Java<br>Intermediate | 100% |
| 3 | Python<br>Intermediate | 100% |

## Tags Distribution

| | | | |
|---|---|---|---|
| Bash | 0% | Arrays and Strings | 0% |
| Medium | 66% | Java | 100% |
| Exceptions | 100% | OOP | 100% |
| Collections | 100% | Python | 100% |

## Questions

Linux/Unix • 0 / 75

| No. | Question | Skill | Score | Code Quality | Time Taken | Status |
|---|---|---|---|---|---|---|
| 1 | Bash: Filename Suffix<br>Coding | Bash (Intermediate) | 0/75 | - | 12 min 52 sec | ⊗ |

Java (Intermediate) • 75 / 75

| No. | Question | Skill | Score | Code Quality | Time Taken | Status |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| 2 | Java: Vending Machine<br>Coding | Java (Intermediate) | 75/75 | - | 45 min 19 sec | ✓ |

Python (Intermediate) • 75 / 75

| No. | Question | Skill | Score | Code Quality | Time Taken | Status |
|---|---|---|---|---|---|---|
| 3 | Python: Book Popularity Tracker<br>Coding | Python (Intermediate) | 75/75 | - | 24 min 32 sec | ✓ |

## 1. Bash: Filename Suffix

Incorrect

Coding Bash Arrays and Strings Medium

### Question description

In file management, systematic renaming of files is essential for clarity and organization, especially with large sets of files.
You are given a list of file names without extensions. Your task is to:

1. Sort the names alphabetically
2. Append an integer to each name based on its order, starting from 1
3. Use an underscore ('_') between the filename and the number

**Example**
*filenames* = ["fileA", "fileC", "fileB"]

Output after sorting and appending the order:

```
fileA_1
fileB_2
fileC_3
```

### Function Description

Complete the function *sort_filenames* in the editor with the following parameters:

*string filenames[n]:* the list of file names

### Outputs

*string[n]:* the list of file names after sorting and adding the suffix

### Constraints

- The length of each file name can be up to 25 characters long.
- The count of file names can be up to 1000.

## ▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *num_filenames*, the size of the array *filenames*.
Each of the next *num_filenames* lines contains a string, *filenames[i]*.

## ▼ SAMPLE CASE 0

### Sample Input 0

```
3
statement_1
filename_2
image_file_3
```

### Sample Output 0

```
filename_2_1
image_file_3_2
statement_1_3
```

### Explanation

The input strings are sorted alphabetically and then appended with an integer denoting their placement in the sorted list. The output shows each string followed by an underscore and the appended integer.

## ▼ SAMPLE CASE 1

### Sample Input 1

```
4
file1
file2
```

```
file3
file4
```

### Sample Output 1

```
file1_1
file2_2
file3_3
file4_4
```

### Explanation

The input lists 4 file names that are already sorted alphabetically. The output appends each file name with an underscore and its sorted position.

## Interviewer guidelines

▼ SOLUTION

### Solution:

```
sort_filenames() {
   local num_filenames=$1
   shift
   local filenames=("$@")
   local sorted_filenames=($(printf "%s\n" "${filenames[@]}" | sort))

   for ((i = 0; i < num_filenames; i++)); do
      sorted_filenames[$i]="${sorted_filenames[$i]}_$((i + 1))"
   done
   echo "${sorted_filenames[@]}"
}
```

## Candidate's Solution

Language used: **BASH**

```
#!/usr/bin/env bash
#
# Complete the 'sort_filenames' function below.
#
# The function is expected to return a STRING_ARRAY.
# The function accepts following parameters:
#  1. INTEGER num_filenames
#  2. STRING_ARRAY filenames
#
```

```

sort_filenames() {
    local num_filenames=$1
    shift
    local filenames=("$@")

}
main() {
    local num_filenames
    read -r num_filenames

    local filenames=()
    for ((i = 0; i < num_filenames; i++)); do
        read -r filename
        filenames+=("$filename")
    done

    sorted_filenames=($(sort_filenames "$num_filenames" "${filenames[@]}"))
    for ((i = 0; i < num_filenames; i++)); do
        echo "${sorted_filenames[$i]}"
    done
}
main
exit 0

```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| TestCase 0 | Easy | Sample | Wrong Answer | 0 | 0.0032 sec | 1.63 KB |
| TestCase 1 | Easy | Sample | Wrong Answer | 0 | 0.003 sec | 1.63 KB |
| TestCase 2 | Easy | Sample | Wrong Answer | 0 | 0.0031 sec | 1.63 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| TestCase 3 | Easy | Hidden | Wrong Answer | 0 | 0.0039 sec | 1.63 KB |
| TestCase 4 | Easy | Hidden | Wrong Answer | 0 | 0.0043 sec | 1.63 KB |
| TestCase 5 | Medium | Hidden | Wrong Answer | 0 | 0.0075 sec | 1.75 KB |
| TestCase 6 | Medium | Hidden | Wrong Answer | 0 | 0.0077 sec | 1.63 KB |
| TestCase 7 | Medium | Hidden | Wrong Answer | 0 | 0.0084 sec | 1.75 KB |
| TestCase 8 | Medium | Hidden | Wrong Answer | 0 | 0.0075 sec | 1.75 KB |
| TestCase 9 | Hard | Hidden | Wrong Answer | 0 | 0.0468 sec | 2.13 KB |
| TestCase 10 | Hard | Hidden | Wrong Answer | 0 | 0.0526 sec | 2.25 KB |
| TestCase 11 | Hard | Hidden | Wrong Answer | 0 | 0.0494 sec | 2.27 KB |
| TestCase 12 | Hard | Hidden | Wrong Answer | 0 | 0.0486 sec | 2.25 KB |

No comments.

## 2. Java: Vending Machine

Correct

Coding Medium Java Exceptions OOP

### Question description

Design a vending machine that serves 3 drinks: "Coke", "Fanta", and "Sprite".

Create a *VendingMachine* class that implements the following methods:

- *void registerDrink(int buttonNumber, Drink drink)*: Registers a type of drink with a particular *buttonNumber* of the vending machine. Here, *Drink* is an interface provided in the locked stub.
- *ServedDrinkSummary(int buttonNumber, int money)*: Returns the Drink that is served when *buttonNumber* is pressed, and an amount of money is entered by a customer.

The class contains two attributes:

- *change*: the units of currency to return to the consumer
- *drink*: the drink requested

There are three potential error conditions:

- If the machine has run out of the drink, an *OutOfStockException* should be thrown with the message "{drinkName} is out of stock".
- If the money the user enters is insufficient to buy the drink, an *InsufficientMoneyException* should be thrown.
- If the drink is out of stock and the user has not entered enough money, throw an *OutOfStockException* with the previously mentioned message.

The *OutOfStockException, InsufficientMoneyException*, and *ServedDrinkSummary* classes are provided in the locked stub.

The locked stub validates the *VendingMachine* class by processing some requests.

### Constraints

- $1 \leq totalNumberOfRequests \leq 10^4$
- $0 \leq buttonNumber \leq 2$
- $0 \leq money \leq 100$
- $0 \leq quantity$ available of each $drink \leq 10^4$

- $0 \leq$ *price* per unit of each *drink* $\leq 10^4$
- No two drinks are assigned the same *buttonNumber.*

## ▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains three integers that represent the *quantity* of *Coke*, *Fanta,* and *Sprite* in the vending machine respectively.
The second line contains three integers that represent the *price* per unit of *Coke*, *Fanta,* and *Sprite* respectively.
The third line contains three integers that represent the *buttonNumbers* assigned to *Coke*, *Fanta,* and *Sprite* respectively.
The fourth line contains an integer, *totalNumberOfRequests .*
The next *totalNumberOfRequests* lines contain the *buttonNumber* pressed and *money* inserted separated by a space.

## ▼ SAMPLE CASE 0

### Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2 1 0    quantity Coke = 2, Fanta = 1, Sprite = 0
5 10 15  price Coke = 5, Fanta = 10, Sprite = 15
0 1 2    buttonNumbers Coke = 0, Fanta = 1, Sprite = 2
4        totalNumberOfRequests = 4
0 20     buttonNumber = 0, money = 20 for the first request
2 20     buttonNumber = 2, money = 20 for the second request
2 5      ...
1 5
```

### Sample Output

```
Vending machine set up
Coke 15
Sprite is out of stock
Sprite is out of stock
Insufficient money
```

### Explanation

- *0 20:* A request to process a can of *Coke* and *20* units of money is provided to the vending machine. *Coke* is in stock and costs *5* units of money so *15* units in change is returned to the consumer. The locked stub prints "*Coke 15".*

- *2 20:* A request to process a can of *Sprite* and *20* units of money is provided to the vending machine. *Sprite* is out of stock so an *OutOfStockException* is thrown, and the stub prints "*Sprite is out of stock".*
- *2 5:* A request to process a can of *Sprite* and *5* units of money is provided to the vending machine. *Sprite* is out of stock and the amount of *money* is insufficient so an *OutOfStockException* is thrown and the stub prints, "*Sprite is out of stock".*
- *1 5:* A request to process a can of *Fanta* and *5* units of money is provided. Since the price of a unit of *Fanta is 10* and it is in stock, an *InsufficientMoneyException* is thrown, and the stub prints "*Insufficient money".*

### Interviewer guidelines

▼ SOLUTION

Solution for Java 7, 8 and 15

```
class VendingMachine{
    HashMap<Integer, Drink> drinkRelatedToButton = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinkRelatedToButton.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)throws OutOfStockException, Insuffi
cientMoneyException{
        ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
        serveDrinkSummary.servedDrink = drinkRelatedToButton.get(buttonPressed);
        serveDrinkSummary.change = money - serveDrinkSummary.servedDrink.getPrice();

        if(serveDrinkSummary.servedDrink.getQuantityLeft() == 0){
            throw new OutOfStockException(serveDrinkSummary.servedDrink.getName() + " is out of st
ock");
        }

        if(serveDrinkSummary.change < 0){
            throw new InsufficientMoneyException();
        }
        return serveDrinkSummary;
    }
}
```

### Candidate's Solution

Language used: **Java 8**

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}

class VendingMachine{
    // buttons 0, 1, 2
    private Map<Integer,Drink> drinkMap = new HashMap<>();
    void registerDrink(int buttonIdx, Drink drink){
        drinkMap.put(buttonIdx, drink);

    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)
        throws OutOfStockException, InsufficientMoneyException{
            Drink drink = drinkMap.get(buttonPressed);
            ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
            if(drink!=null){
                if(drink.getQuantityLeft() <= 0){
                    throw new OutOfStockException(drink.getName() +" is out of stock");
```

```
                }
                if(drink.getPrice() > money){
                    throw new InsufficientMoneyException();
                }
                serveDrinkSummary.servedDrink = drink;
                serveDrinkSummary.change = money - drink.getPrice();


            }
            return serveDrinkSummary;
    }
}

class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
```

```java
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
```

```
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
```

```
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| TestCase 0 | Easy | Sample | Success | 1 | 0.0679 sec | 23.8 KB |
| TestCase 1 | Easy | Hidden | Success | 6 | 0.0513 sec | 23.9 KB |
| TestCase 2 | Easy | Hidden | Success | 6 | 0.0594 sec | 26 KB |
| TestCase 3 | Easy | Hidden | Success | 6 | 0.0718 sec | 25.9 KB |
| TestCase 4 | Medium | Hidden | Success | 10 | 0.0699 sec | 25.8 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| TestCase 5 | Medium | Hidden | Success | 10 | 0.1438 sec | 41.1 KB |
| TestCase 6 | Medium | Hidden | Success | 10 | 0.1465 sec | 40.2 KB |
| TestCase 7 | Hard | Hidden | Success | 13 | 0.1021 sec | 39.7 KB |
| TestCase 8 | Hard | Hidden | Success | 13 | 0.0983 sec | 38.9 KB |

No comments.

## 3. Python: Book Popularity Tracker

Correct

Coding Medium Collections Python

### Question description

Implement a Python application for managing a bookstore's inventory. The application should allow adding books, finding highest-rated books, and recommending books by genre.

There are two classes:

1. *Book* class with attributes:
   - *title (string)*: the book's title
   - *genre (string)*: the book's genre
   - *rating (float)*: the customer rating
2. *LibraryManagement* class with methods:
   - *add_book(Book book)*: adds a new book to the inventory
   - *get_highest_rated_books(int x)*: returns the *x* highest-rated books, sorted by *rating* (descending) and *title* (descending) for ties

- *recommend_top_book_by_genre(string genre)*: returns the highest-rated book in the specified genre, or None if no books exist in that genre

The code for reading input, calling methods, and producing output is already provided.

### Constraints

- The number of recommended books is less than or equal to the number of books in the book list.

### ▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the number of books.

Each of the next *n* lines contains a comma-delimited list of *title, genre,* and *rating*.

The next line contains an integer, the number of books to recommend.

The last line contains a string, the genre to recommend from.

### ▼ SAMPLE CASE 0

#### Sample Input For Custom Testing

```
3
Python Programming,Programming,4.5
Data Science Essentials,Data Science,3.1
C++ Programming,Programming,4.3
1
Programming
```

#### Sample Output

```
Highest Rated Book: Python Programming, Rating: 4.5
Top Recommended Book in Programming Genre: Python Programming, Rating: 4.5
```

#### Explanation

Here the number of recommended books is 1.

The specified genre is "Programming".

### ▼ SAMPLE CASE 1

#### Sample Input For Custom Testing

```
5
Python Programming,Programming,4.5
Data Science Essentials,Data Science,3.1
C++ Programming,program,4.3
```

```
Algorithm,Algorithm,2.9
Java Programming,Programming,4.7
3
Programming
```

### Sample Output

```
Highest Rated Book: Java Programming, Rating: 4.7
Highest Rated Book: Python Programming, Rating: 4.5
Highest Rated Book: C++ Programming, Rating: 4.3
Top Recommended Book in Programming Genre: Java Programming, Rating: 4.7
```

### Explanation

Here the number of recommended books is 3.

The specified genre is "Programming".

## Interviewer guidelines

▼ SOLUTION

```
class Book:
    def __init__(self, title, genre, rating):
        self.title = title
        self.genre = genre
        self.rating = rating

class LibraryManagement:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def get_highest_rated_books(self, x):
        sorted_books = sorted(self.books, key=lambda x: (x.rating, x.title), reverse=True)
        return sorted_books[:x]

    def recommend_top_book_by_genre(self, genre):
        genre_books = [book for book in self.books if book.genre == genre]
        if not genre_books:
            return None
        return max(genre_books, key=lambda x: (x.rating, x.title))
```

**Candidate's Solution**

Language used: **Python 3**

```python
#!/bin/python3

import math
import os
import random
import re
import sys

class Book:
    def __init__(self, title, genre, rating):
        self.title = title
        self.genre = genre
        self.rating = rating
        pass

class LibraryManagement:
    def __init__(self):
        self.books = []
        pass

    def add_book(self, book):
        self.books.append(book)
        pass

    def get_highest_rated_books(self, x):
        sorted_books = sorted(self.books, key= lambda book : (book.rating, book.title),reverse=True)
        return sorted_books[:x]
        pass

    def recommend_top_book_by_genre(self, genre):
        genre_books = [
            book for book in self.books
            if book.genre.lower() == genre.lower()
        ]

        if not genre_books:
            return None

        return max(genre_books, key = lambda book: book.rating)
```

```python
        pass
def main(path) -> None:
    fptr = open(path, 'w')
    num = int(input())
    library = LibraryManagement()
    for i in range(num):
        title, genre, rating = input().split(",")
        library.add_book(Book(title, genre, float(rating)))

    recommended_count = int(input())
    genre = input()

    highest_rated_books = library.get_highest_rated_books(recommended_count)
    book_by_genre = library.recommend_top_book_by_genre(genre)

    for book in highest_rated_books:
        fptr.write(f"Highest Rated Book: {book.title}, Rating: {book.rating}\n")

    if book_by_genre:
        fptr.write(f"Top Recommended Book in Programming Genre: {book_by_genre.title}, Rating: {book_by_genre.rating}\n")
    else:
        fptr.write("No books found in the specified genre.\n")
    fptr.close()

if __name__ == "__main__":
    env_var = "OUTPUT_PATH"
    path = os.environ[env_var] if env_var in os.environ.keys() else "/dev/stdout"
    main(path)
```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| Testcase 0 | Easy | Sample | Success | 1 | 0.0274 sec | 10.8 KB |
| Testcase 1 | Easy | Sample | Success | 1 | 0.0283 sec | 10.8 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| Testcase 2 | Easy | Sample | Success | 1 | 0.0276 sec | 10.8 KB |
| Testcase 3 | Easy | Hidden | Success | 5 | 0.0246 sec | 10.8 KB |
| Testcase 4 | Medium | Hidden | Success | 7 | 0.0257 sec | 10.8 KB |
| Testcase 5 | Medium | Hidden | Success | 10 | 0.0246 sec | 10.6 KB |
| Testcase 6 | Medium | Hidden | Success | 10 | 0.0263 sec | 10.6 KB |
| Testcase 7 | Hard | Hidden | Success | 15 | 0.0327 sec | 10.8 KB |
| Testcase 8 | Hard | Hidden | Success | 25 | 0.0273 sec | 10.8 KB |

No comments.